BeyondGrip: Physics-Informed Control Recovery for Racecars

Avery Chan, Dongheon Lee, Junyoung Kim, Christopher Tan

***Abstract*—Autonomous racing at the limits of friction poses significant safety challenges, particularly when vehicles encounter unexpected loss-of-control events. While standard Deep Reinforcement Learning (DRL) agents can master nominal racing lines, they often fail to generalize to high-slip recovery scenarios due to a lack of underlying physical understanding. This paper presents *BeyondGrip*, a physics-informed reinforcement learning framework designed to recover racecars from severe instability. We integrate vehicle dynamics directly into the learning process by augmenting the state space with tire slip ratios and aligning torque, and by designing a reward function grounded in friction limits and load transfer. Using the high-fidelity *Assetto Corsa* simulator, we compare our physics-informed TD3 agent against standard SAC and TD3 baselines. Our results demonstrate that while baseline methods degrade rapidly in Out-of-Distribution (OOD) scenarios, the proposed physics-aware agent exhibits superior robustness, achieving higher recovery rates and significantly lower crash severity under varying speeds and surface conditions. This suggests that explicit physical inductive biases are essential for developing safe, generalizable autonomous racing controllers.**

I. Objective

This project aims to develop autonomous agents that are able to recover a race car from situations in which tire grip is temporarily lost, a scenario that poses significant challenges for both human drivers and autonomous control systems. Rather than relying solely on standard reinforcement learning approaches, this work incorporates physics-informed techniques during the learning process to better capture vehicle dynamics. The focus is placed on recovery during cornering, acceleration, and braking, where tire grip becomes unpredictable and precise control is required. These dynamics-aware recovery behaviors are learned within the high-fidelity *Assetto Corsa* simulator.

II. Motivation

While *Assetto Corsa* offers a high fidelity physics engine, most reinforcement learning agents rely on the simulator only for reward feedback rather than physical understanding. Earlier research, using learning from demonstration shows weakness in out-of-distribution cases, such as oversteer, understeer or traction loss. Humans use feedback from tire-road interaction through the steering wheel to regain control of the car. Motivated by this observation, this project integrates vehicle dynamics, physical models, and physics-based reward design into the reinforcement learning framework. By doing so, the agent is encouraged to develop a stronger connection between vehicle behavior and road interaction, enabling more robust recovery control in challenging driving conditions.

A. Key Differences From Prior RL Work

We view recovery as one component of a hierarchical racing stack. A nominal driving controller governs standard operation during racing, optimizing for high speed maneuvers. A supervisor monitors indicators of instability and triggers a dedicated recovery policy. Once stability is restored, control is smoothly handed back to the nominal policy. Unlike many previous recovery RL formulations which focus only on reaching *any* safe state, our goal is to return the vehicle to a *high-quality* safe state that supports rapid resumption of nominal racing. Intuitively, a successfully recovery policy need not only stop loss of control, but it should also smoothly re-enter a safe set with low slip, small track deviation, and a favorable exit speed.

Finally, while the focus of this report is recovery, we also trained a nominal racing policy separately (MPC and RL) and validated that the recovery controller can hand control back once stability is restored. In other words, our system is designed to operate as a two policy stack, but the experimental results in this paper emphasize the recovery policy and its robustness under OOD loss-of-grip conditions.

A natural alternative is to train a single "robust RL" policy that handles both nominal racing and recovery. We do not pursue this approach because the two modes impose substantially different objectives and control requirements. In our tests, nominal racing favors smooth, incremental corrections around a stable operating point, whereas recovery requires short-horizon, aggressive stabilization actions under highly nonlinear slip dynamics. In our implementation, these differences are reflected even at the action interface: nominal driving is most effective with a relative/incremental control parameterization, while recovery benefits from an absolute control parameterization to enable immediate counter-steer and decisive throttle/brake interventions. Empirically, we found that relative-action recovery policies were difficult to train and failed to reliably converge in the recovery regime. This motivates our explicit hierarchical separation rather than forcing one robust policy to trade off between incompatible regimes.

III. Background

A. Reinforcement Learning Formulation

We model our problem as a Partially Observable Markov Decision Process (MDP) which is defined by the following seven tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \mathcal{O}, \mathcal{Z}, \gamma)$ where $\mathcal{S}$ is the state of the simulator (current car position, velocity, heading, etc.), $\mathcal{A}$ are the valid actions that can be taken in that state (steering, braking, accelerating controls), $\mathcal{P}$ is the transition function

encoded in the simulator, $\mathcal{R}$ is our reward model, $\mathcal{O}$ is our observation space, $\mathcal{Z}$ is the observation function (that for any state-action 2-tuple offers an obscured representation of the underlying MDP), and γ is the temporal discount factor. For convenience, we simplify our representations to use states rather than observations (i.e. $\pi(s)$ rather than $\pi(o)$) since this more closely conforms to existing notation used specifically in reinforcement learning.

Our objective is to learn an optimal policy $\pi(a|s)$ for all states that optimally recovers from tire grip loss.

$$\pi^* \in \arg\max_{\pi} \mathbb{E}_{\substack{s_0 \sim \mu_0(\cdot) \\ a_t \sim \pi(\cdot|s_t) \\ s_{t+1} \sim \mathcal{P}(\cdot|s_t, a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, a_t) \right] \quad (1)$$

$$\text{with } s \in \mathcal{S},\ a \in \mathcal{A}.$$

Since we do not know $\mathcal{P}$ exactly, or at least we don't have a mathematical model that can easily be derived from the simulator, we are in the learning domain.

We calculate, or at least closely approximate π^* through learning. To do this, we reparameterize π^* as π_θ. We simplify and represent a series of states, actions, and rewards using a single new mathematical object: a trajectory. The distribution over trajectories created when using policy π_θ is represented using notation $P_\theta(\cdot)$. The standard method, to carry out the above optimization is then REINFORCE which uses Gradient Ascent, where the derivative is calculated as follows:

$$\nabla_\theta \mathbb{E}_{\tau \sim P_\theta(\cdot)} \left[\mathcal{R}(\tau) \right] \quad (2)$$

$$\mathbb{E}_{\tau \sim P_\theta(\cdot)} \left[\left(\sum_{t=0}^{\infty} \nabla_\theta \log \pi_\theta(a_t|s_t) \right) \mathcal{R}(\tau) \right] \quad (3)$$

The algorithms we use build upon this method: Soft Actor Critic (SAC) and Twin Delayed DDPG (TD3). We offer an intuitive explanation of the major differences in these algorithms in our appendices, that we skip for brevity.

B. Physics-Informed Techniques

In machine learning, it is common to look for a variety of supervision signals for training tasks to incorporate external world knowledge in our training tasks. Since scientists have developed incredibly intricate and delicate theories of our world, we can use this as a domain-specific regularization technique that enables models to ignore solutions that simply do not respect the laws of physics. Most commonly, we can replace, in supervised learning scenarios, the simple loss with the following:

$$\mathcal{L} = \mathcal{L}_{\text{simple}} + \alpha \mathcal{L}_{\text{physics}} \quad (4)$$

These techniques often assist with data sparsity of the base loss, ensure safe exploration, increase simulator accuracy, increase convergence speed, as well as more transparently describe the model. RL benefits especially from these techniques due to the fact that most environments in which RL is deployed include some aspect of physics whether in the simulator or in the real world. RL is also often applied to domains in which real data is scarce; these scenarios are those such that RL was a last resort to begin with.

Three major methods of Physics-Informed RL include Observational Biasing, Learning Biasing, and Inductive Biasing [1]. The first involves using the underlying physics to create additional data for training often across modalities. The second involves manipulating rewards and losses as shown above. The third involves modifying the underlying architecture of the neural networks to make unphysical solutions unrepresentable using any such class of functions.

One major technique of Observational Biasing that we employ is State Design; Instead of requiring the policy network to infer key physical relations, we explicitly compute physically meaningful features and append them to the observation. This introduces redundant state information that improves learnability by making relevant dynamics directly observable. While such features are deterministic functions of the original non-augmented observation and therefore do not increase representational capacity in theory, in practice exposing dynamics-aligned coordinates reduce sample complexity and failure modes in which the learned policy violates basic vehicle dynamics behavior.

For example, say that our state is described through mass m and velocity v. To increase the information available to our networks, we introduce momentum as a third state variable that makes learning about $p = mv$ much easier. While expressive networks can represent many functions of (m, v), universal approximation is a representational result under many assumptions and is not a guarantee that training will reliably discover the desired mapping. Explicit physics-derived features (e.g., $p = mv$) can nonetheless improve sample efficiency and stability by biasing learning towards a known relevant structure.

The second major technique we used is Reward Design under Learning Biasing. As the name suggests we reward agents that more closely follow estimate physical laws or recommend actions that are likely to do so through additional loss or reward terms.

IV. Literature Review

A. Driver Agents

A large number of former works have focused on training racing agents that are able to drive. Some works focused on imitation learning using human experts [5]. For autonomous driving, others specifically use physics to restrict action spaces: many reinforcement learning setups occasionally reformulate actions as being state dependent since not all actions remain available at all states $\mathcal{A}(s)$; in many works though, the standard approach is to retain a static action space and negatively reward impossible actions; the authors of [7], however, restrict the actions that likely would, according to physics, lead to slipping and not through rewards shaping but action mapping where a unconstrained virtual π_V is mapped to a constrained real π_R through a numerical approximation of the physics derivations. Yet others use model-based RL techniques to learn

the transition function $\mathcal{P}$ for further planning or closed form RL solutions [2]. Finally, classic PINN-style physics losses and reward shaping used for PIRL simpler than ours have also been tried [3].

Our work utilizes these ideas and we try to further build on these techniques, trying to improve their results. The main work of this paper adds another direction in which methods for driving specifically can be expanded upon and theorizing if this technique may work well with any others.

B. Physics-Informed Reinforcement Learning

As for work that is similar in type though different in objectives, the survey [1] emphasizes a wide range of PIRL techniques, both the reward and state shaping that we employ, and a host of others. For non-racing related tasks, the following ideas have already been tested: modifying the action space, modifying the policy or value functions, and modifying the simulator for increased physical accuracy.

The survey describes a complex interplay of information (differential equations, constraints, parameters, data, etc.), methods (action, state, reward, policy/value, and environment shaping). It also offers the above classifications of biasing (Observational Biasing, Learning Biasing, and Inductive Biasing) and learning architectures (safety filter, differentiable simulator, etc.).

The overall RL training pipeline can change in the above ways, first identify what *information* is available: laws in the forms of differential equations, extra data, etc.; then we look at the section of the *pipeline* to modify using our biasing techniques, then the specific section of the RL *setup* to change, right before changing it in one of the established ways i.e. *learning architectures*. They also offer that the pipeline itself might change (the formulation of the problem, etc.).

Our work uses specific ideas within this mountain of ideas, and we leave open the rest of the methods for consideration in future work.

V. Methodology

A. Overview

Our central objective is to create agents that are able to recover from slipping. We use the *Assetto Corsa* simulation environment to train our reinforcement learning agents. For additional support, we use a simulator wrapper that offers an OpenAI Gym Environment endpoint [5] that we issue actions and read states and rewards from. We run our training on a machine with a 6-core CPU, an RTX A4000 GPU, 16 GB of RAM, and 200 GB of storage.

We implemented the TD3 algorithms alongside the existing SAC baseline algorithm.

In order to prime the simulator for specifically cases in which the car slips, and from which we try to recover, we use a standard model predictive control (MPC) controller which, as its name suggests, uses a model of the environment to model what happens in the future and uses traditional optimization techniques, to reach a certain speed before we induce slipping conditions after which we start our training episodes.

We introduce the following modifications:

- *PIRL:* Expanded state (yaw, pitch, per-wheel roll, self-alignment torque, slip ratios, wheel loads); computed steering from slip angles + alignment torque; enforced throttle/brake via weight transfer + per-wheel slip to maximize grip/stability.
- *Training:* Adjusted termination and non-physics rewards; varied replay buffer size; used absolute action controllers rather than relative action policies; added TD3 action-smoothing regularization.
- *Testing:* Built ID/OOD suite with modified/faulty sensors; stratified sampling over velocities; added unrecoverable cases to assess crash-severity mitigation when recovery is impossible.

B. Slip Induction and Episode Initialization

To consistently train slip recovery behaviors, we explicitly design each training episode to begin from a controlled yet unstable vehicle state. At the start of every episode, a MPC controller drives the vehicle along the reference trajectory and accelerates it to a target speed selected via stratified sampling over the training speed range (60–100 km/h).

We partition this range into eight fixed width bins and cycle through bins across each episode to ensure uniform coverage of realistic speed regimes. Within each selected bin, the target speed is sampled uniformly at random.

Once the target speed is reached, a slip is induced by applying a short-duration steering disturbance, referred to as a *steering flick*. The flick parameters feint duration, counter-steer duration, counter-steer magnitude, and intensity are sampled from truncated normal distributions to concentrate most episodes around nominal slip intensities while still producing diverse recovery conditions. In addition, we intentionally allocate a small fraction of samples to the extreme ends of these bounds to generate effectively unrecoverable cases, enabling evaluation of whether the agent learns to mitigate crash severity even when full recovery is infeasible. This disturbance rapidly increases the rear tire slip angle and pushes the vehicle into an unstable regime representative of loss-of-control events observed in aggressive driving.

Control authority is transferred from the MPC controller to the RL agent immediately after the counter-steer phase ends, at which point the vehicle is typically in a high slip regime with rear slip angle on the order of 5–20°, ensuring that training focuses on active slip mitigation rather than nominal trajectory tracking.

C. State and Action Space Design

Effective slip recovery requires the agent to reason about vehicle stability, tire dynamics, and control authority under highly nonlinear conditions. To this end, we use an augmented state space from [5] that captures both kinematic and dynamic properties of the vehicle, with particular emphasis on quantities directly related to loss-of-control events.

Together, these features allow the policy to distinguish between nominal driving, recoverable slip, and severely unstable

states without relying on implicit inference through reward signals alone.

Because recovery is partially observable and highly sensitive to transient dynamics, we additionally augment the observation with previously applied control action by the agent. To capture short horizon temporal effects, we employ history stacking by concatenating the most recent 3 observations into a single observation vector, allowing a feedforward policy to approximate memory, without have a recurrent state.

The action space consists of three continuous control inputs: steering, throttle, and braking. Steering commands are normalized to the range $[-1, 1]$, while throttle and brake commands lie in $[0, 1]$.

We experimented with two action parameterizations: absolute controls, where the policy outputs steering, throttle, and brake commands directly each timestep, and relative controls where the policy outputs increments that are accumulated at some tunable maximum rate into the applied controls. Although we found that relative actions can be advantageous in nominal driving due smoother control updates, they consistently failed to converge in our induced slip recovery scenarios. We observed that recovery episodes begin from a highly precarious state, if the policy does not rapidly counter-steer to correct heading and stabilize slip within a short horizon, the vehicle quickly becomes unrecoverable and the episode terminates due to loss of control (e.g, leaving the track or colliding with the wall). In the most severe cases, the slip angle is increased to such a degree that steering authority is effectively and additional steering produces little to no change in vehicle heading. Under the parameterization of relative control, the required large corrective steering must be produced through multiple accumulation steps. In practice, exploration noise and action accumulation caused incremental commands to average out, resulting in insufficient steering authority.

We attempted to remedy this failure by improving exploration by including temporally correlated noise such as Ornstein–Uhlenbeck noise [6] as commonly used in DDPG [4] and non-independent action perturbations as well action duplication (repeating actions over multiple simulator steps), but these modifications failed to yield recovery. Consequently, we adopt absolute actions for recovery to ensure immediate control authority. We note that absolute actions can reduce smoothness and introduce jitter. We address this separately via regularization and other action smoothing mechanisms.

This divergence suggest that nominal driving and slip recovery demand fundamentally different control representations. This motivates a hierarchical RL architecture where our nominal policy uses absolute actions to deliver rapid, high authority interventions under extreme slip.

D. Termination Criteria and Recovery Definition

Defining a meaningful termination condition is critical in slip recovery tasks, as naive criteria can lead to degenerate solutions that exploit transient dynamics. In preliminary experiments, we observed that agents could momentarily reduce slip angle through aggressive braking or steering corrections while remaining in an unstable heading configuration. Such behaviors often triggered premature success signals despite the vehicle being unable to maintain stability or remain on track.

To mitigate this issue, we define a successful recovery as maintaining a slip angle below 5 degrees for a continuous duration of 1.5 seconds. This temporal requirement ensures that recovery corresponds to a genuinely stabilized vehicle state rather than a brief fluctuation caused by control impulses. The slip threshold of 5 degrees was selected empirically as a boundary below which the vehicle consistently exhibits stable handling behavior across a wide range of speeds, while still allowing sufficient tolerance for sensor noise and minor oscillations.

In addition to slip stabilization, we require the lateral deviation from the reference trajectory to remain within 3 meters throughout the recovery window. This constraint prevents policies that achieve low slip angles at the expense of large heading errors or off-track excursions, which would be undesirable in practical racing scenarios. We also considered smoothing-based alternatives such as exponential moving averages for slip measurements; however, enforcing a sustained low-slip condition over time was found to be a more interpretable and robust criterion during training.

E. Reward Design

1) Termination Rewards:

- On success (recovered from slip):

$$r = r_{\text{base}} + r_{\text{time}}(T) + r_{\text{speed}}(v) + r_{\text{gap}}(d) \tag{5}$$

  Where T is time to completion, v is speed, and d the overall distance to a precomputed reference line
- On failure (crash):

$$r = -p_{\text{base}} - p_{\text{severity}}(s, d, v)\, \exp(-\alpha T) \tag{6}$$

  Where s is the slip. Note this highly non-Markovian reward decays penalties (rewards) models that survive for longer.(full expression in Appendix).

2) Non-physics Rewards: : During recovery we apply a per-step shaping reward of the form

$$r_t \;=\; \text{clip}\Big(\beta\Big(a_t - p_s(T) - p_d(T) + i_t\Big),\, r_{\min},\, r_{\max}\Big) \tag{7}$$

Here, a_t is an "alive" bonus for remaining in control, i_t is an improvement term, and $p_s(t)$ and $p_d(t)$ penalize rear slip and lateral deviation (full expression in Appendix VIII-A). Both penalties are computed from normalized signals, capped to avoid outliers, and applied quadratically, so large deviations are disproportionately discouraged. In addition, the slip and gap penalties are exponentially time-decayed within an episode, emphasizing rapid stabilization early while still assigning partial credit later in long episodes. The improvement term provides dense feedback when the vehicle returns toward the reference line by rewarding reductions in gap between consecutive steps. Finally, a_t is increased in near-stable regimes (small slip and small gap), biasing the policy toward entering and maintaining stable recovery states.

3) Physics Rewards: To guide the agent toward robust oversteer recovery, we implemented a physics-based reward function that incentivizes optimal vehicle dynamics through four mechanical objectives:

- *Load Transfer:*
  Rewards shifting vertical load (F_z) to the rear axle during oversteer. This implicitly encourages throttle modulation to shift weight rearward, increasing the friction budget at the sliding rear tires to accelerate grip restoration.
- *Slip Ratio Control:*
  Constrains the rear tire slip ratio (κ) to a target of 1.5% using a Gaussian reward. This prevents wheel spin saturation, ensuring the agent operates at the peak of the friction curve to maximize lateral stability.
- *Counter-Steering Alignment:*
  Rewards aligning the steering angle (δ) with the vehicle's sideslip angle (β). This minimizes front tire scrub and kinematically orients the vehicle to arrest yaw rotation, enforcing the fundamental counter-steering maneuver.
- *Aligning Torque Compliance:*
  Mitigates "snap oversteer" by rewarding steering power ($P_{steer} = M_z \cdot \dot{\delta}$) that aligns with the self-aligning torque (M_z). This encourages the agent to yield to stabilizing forces and smoothly unwind the steering wheel as grip returns, mimicking the haptic feedback used by expert drivers to dampen yaw oscillations.

$$r_{\text{physics}} = r_{\text{load}} + r_{\text{slip ratio}} + r_{\text{align}} + r_{\text{compliance}} \tag{8}$$

The various hyperparameters definitions above can be located in the Appendices.

F. Experience Replay Buffers

All off-policy agents are trained using an experience replay buffer to improve data efficiency. The buffer stores transitions of the form $(s_t, a_t, r_t, s_{t+1}, \text{done}_t)$ collected under exploration during training. Each update step, a minibatch is sampled uniformly for gradient updates. To improve short-horizon credit assignment, we use a 3-step return. The reward is

$$R_t^{(3)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2}, \tag{9}$$

We found that this creates a dense enough learning signal, while keeping the horizon short enough to avoid excessive variance. With a control rate of 25 Hz, $n = 3$ corresponds to a horizon of roughly 0.12 s. Finally, an ablation on replay buffer size indicates that increasing buffer capacity improves convergence rate and robustness, highlighting replay capacity as a critical hyperparameter in this domain.

G. Evaluation Protocol and Test Scenarios

1) Evaluation Mode and Episode Phases: All evaluations run using purely exploitative actions where the agent acts deterministically without exploration noise. Each episode follows a three phase structure as noted in Section V-B

2) Reported Metrics: We report recovery success rate (fraction of episodes recovered) with Wilson 95% confidence intervals. For episode-level analysis we summarize key telemetry using median and interquartile range (IQR). For OOD stress tests, we additionally report a terminal crash severity score and its weighted component contributions (slip, gap, speed), including both per-agent failure-conditional summaries and matched hard-set summaries on a fixed episode subset defined by a reference agent's failures.

3) In Distribution (ID) Testing: ID evaluation uses the same scenario generation process as training including the stratified speed coverage, but runs the learned policy deterministically. We evaluate 20 episodes per speed bin, yielding $20 \times 8 = 160$ total ID episodes.

4) Out of Distribution (OOD) Generalization Testing: We evaluate generalizability along three OOD axes, varied independently:

- *High Speed:* target speed fixed above the training range (110 km/h).
- *High Intensity:* steering flick intensity increased beyond nominal levels.
- *High Counter Weight:* counter ster magnitude multiplier increased beyond nominal levels.

Each OOD axis/test is evaluated over 36 episodes

5) In Distribution Robustness Testing: We additionally evaluate robustness using a fixed representative slip configuration (nominal flick parameters at 80 km/h).

- *Observation perturbation (Yaw scaling):* yaw rate observations scaled by a multiplicative factor drawn from a truncated normal distribution once per episode.
- *Action perturbation (steering authority reduction):* steering clipped to 70 % of the full range.
- *Combined perturbation:* yaw scaling and steering clipping applied simultaneously.

In addition to the factor isolated OOD tests we include a hard OOD tests that combines all of the above distribution shifts simultaneously at their worst-case tested value. Each robustness test class is evaluated over 50 episodes.

6) Reproducibility: To ensure fair and repeatable comparisons across algorithms and checkpoints, we generate evaluation episodes using a fixed random seed. This yields a deterministic sequence of scenario parameters (e.g., speed-bin selection and sampled flick parameters) across runs, so differences in performance reflect policy changes rather than stochastic test variation. From our experience, given slip parameters and a deterministic policy, the dynamics of the slip and its recovery are relatively deterministic.

H. Algorithm Modifications

We evaluate four agent configurations that differ in (i) the underlying RL algorithm, (ii) replay buffer capacity, and (iii) whether the reward is physics-informed. Two baselines use the standard TD3 algorithm with a *non-physics-based* reward and differ only in replay buffer size: TD3_30k uses a 30,000-transition replay buffer, while TD3_60k uses a 60,000-transition replay buffer. We also include a SAC baseline

trained with the same non-physics-based reward and a 60,000-transition replay buffer to compare TD3 against a stochastic off-policy actor–critic method under matched replay capacity. Finally, our proposed method, TD3_Phys, uses TD3 with the same 60,000-transition replay buffer but replaces the baseline reward with a physics-informed reward designed to better reflect slip and stability-related vehicle dynamics. All algorithms use identical termination criterion and rewards. Unless otherwise noted, all other hyperparameters and training settings are held constant to isolate the effects of replay capacity and physics-informed reward design.

In our TD3 implementation, we augment the standard actor objective with two auxiliary terms that improve control smoothness and lateral stability during recovery. Importantly, these terms are applied to the policy loss (not the reward) so that they shape the learned action outputs without changing the critic targets or redefining the task's return.

Let the base TD3 actor loss be

$$\mathcal{L}_{\text{base}} \;=\; -\mathbb{E}_{s\sim\mathcal{D}}\big[\,Q_\theta\big(s,\pi_\phi(s)\big)\big]\,. \tag{10}$$

We optimize the total actor loss

$$\mathcal{L}_\pi \;=\; \mathcal{L}_{\text{base}} + \underbrace{\alpha\,\mathcal{L}_{\text{smooth}}}_{\text{auto-regulated action smoothing}} - \underbrace{\beta\,\mathcal{L}_{v_{\text{lat}}}}_{\text{lateral regularizer}} \tag{11}$$

where α and β control the contribution of each auxiliary term.

a) Auto-regulated action smoothing.: We penalize large action changes using the previous action stored in the observation (from replay). Let Δa denote this action difference; we compute the raw smoothness penalty

$$\mathcal{L}_{\text{smooth,raw}} \;=\; \mathbb{E}\big[\|\Delta a\|_2^2\big]\,. \tag{12}$$

To make this penalty scale-invariant we then scale it relative to the current base actor loss using a target ratio ρ (e.g., $\rho = 0.03$):

b) Lateral stability regularizer.: In addition, we include a small term based on signed gap (track offset) and lateral velocity v_{lat}. Let d denote the signed gap and define a clipped, normalized lateral velocity $\hat{v}_{\text{lat}}$ We define

$$\mathcal{L}_{v_{\text{lat}}} \;=\; -\mathbb{E}[\tanh(-d\,\hat{v}_{\text{lat}})]\,, \tag{13}$$

and include it with a small weight β (e.g., $\beta = 0.02$). Intuitively, this term biases the policy toward actions that reduce lateral motion in a direction consistent with decreasing track deviation.

Overall, this yields smoother, less oscillatory recovery actions while preserving the underlying TD3 objective of maximizing predicted return.

"SAC was trained with standard hyperparameters, while the target entropy was annealed to -0.1.

VI. Outcomes and Discussion

A. Recovery Success Over Training

In Figure 1 we report learning curves showing recovery success rate versus training environment steps to quantify progress and to show convergence.

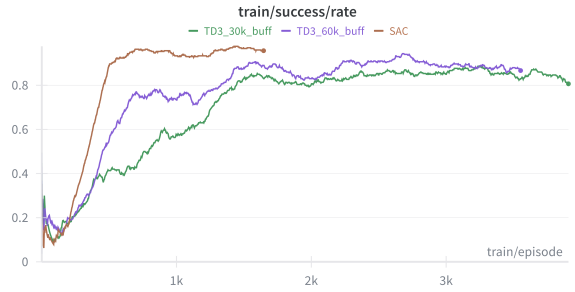

Fig. 1. Learning curves of training success rate for TD3 under different replay buffer capacities, with SAC as a baseline.

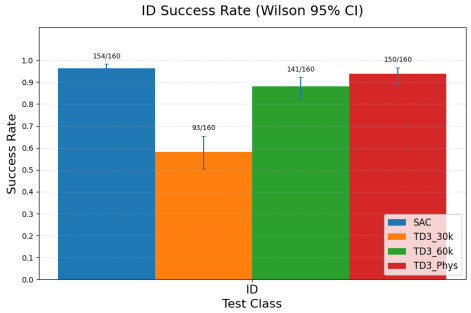

Fig. 2. Success rate of In Distribution (ID) test cases over all algorithms

B. Aggregate Performance Overview

Figure 2 reports in-distribution (ID) recovery success rates with Wilson 95% confidence intervals over 160 episodes per agent. All methods achieve high ID performance except TD3_30k, which underperforms substantially, indicating that limited training produces a policy that is not yet reliably stable even on nominal scenarios. Among the stronger baselines, SAC attains the highest ID success rate, with TD3_Phys and TD3_60k slightly lower. The confidence intervals suggest that SAC and TD3_Phys are close on ID, while TD3_60k is noticeably less reliable. Overall, these results confirm that TD3_Phys preserves strong nominal recovery performance while improving robustness in later OOD analyses.

Figure 3 summarizes out of distribution recovery success

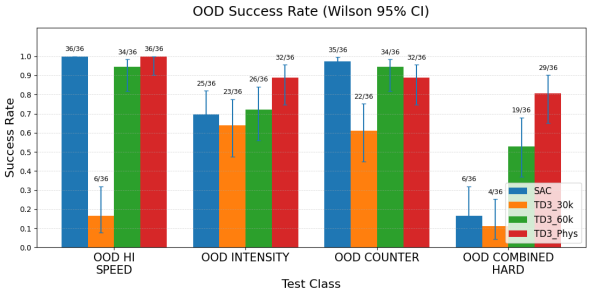

Fig. 3. Success rate of Out of Distribution (OOD) test cases over all algorithms

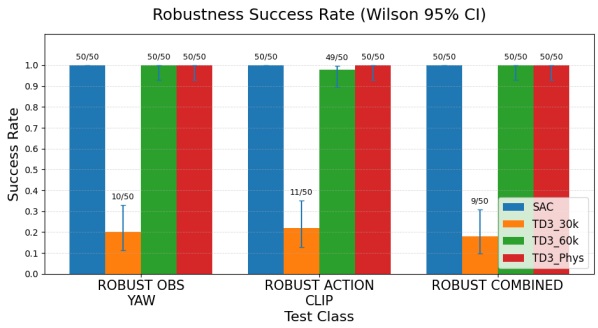

Fig. 4. Success rate of Robust and In Distribution test cases over all algorithms

TABLE I
IN-DISTRIBUTION METRICS AS A % IMPROVEMENT OVER SAC
↓ MEANS LOWER IS BETTER, ↑ MEANS HIGHER IS BETTER, W/ SAC AT 0%

metric	$TD3_{60k}$	$TD3_{Phys}$
Final Gap ↓	-20.5	+199.1
Final Speed ↑	-29.9	+10.0
Max Gap ↓	-3.7	+29.9
Max Rear Slip Angle (deg) ↓	-0.9	+26.6
Max Yaw Rate ↓	-3.3	+20.9
Mean Gap ↓	-13.4	+15.7
Mean Rear Slip Angle (deg) ↓	-2.7	+19.8
Mean Speed ↑	-11.6	+7.8
Min Rear Dy ↑	+1.6	-2.6
Recovery Steps ↓	+1.2	+38.6
Recovery Time (sec) ↓	+1.2	+38.6

rates across the aforementioned four test classes. On single factor shifts, performance is strong for baseline SAC and TD3 (small replay buffer TD3 has poor performance across the board). However, on the the more challenging slip perturbations, it exposes a clear robustness gaps. The strongest separation occurs on the compound shift. Both TD3 algorithms outperform SAC, but specifically this indicates that the physics informed reward substantially improves generalization when multiple stressors co-occur, which better reflects realistic recovery conditions than any single factor shift.

Figure 4 shows that all adequately trained agents (SAC, TD3_60k, and TD3_Phys) perform well across all robustness tests, indicating strong resilience to observational yaw noise, action clipping, and their combination. This result is somewhat surprising. In practice, across a vast majority of episodes, all agents use more than 70% of the available steering range (the clipping threshold). This suggests that their recovery behavior relies on the direction and timing of steering in relation to the observed state rather than a fine-grained memorized script utilizing the full steering range.

C. In Distribution Analysis

We use SAC as a baseline and compare our two best methods against it over 160 runs for in distribution cases. The median results over those runs are shown in Table I.

We see that our methods improve over SAC in terms of increasing aggressiveness, with a much larger final speed

TABLE II
TOTAL RELATIVE RECOVERY TIME COMPARED TO SAC (%)
LOWER IS BETTER

$TD3_{60k}$	$TD3_{Phys}$
33.5819	5.9772

TABLE III
WEIGHTED COMPONENT SEVERITIES ON `OOD_COMBINED_HARD`, COMPUTED EXACTLY AS IN THE TERMINAL REWARD. EACH ENTRY REPORTS MEDIAN AND INTERQUARTILE RANGE (Q1–Q3)

Agent	n	1.2· slip	0.5· gap	0.25· speed
SAC	36	0.067 [0.001–0.287]	0.053 [0.048–0.089]	0.194 [0.194–0.194]
TD3_60k	36	0.000 [0.000–1.013]	0.000 [0.000–0.053]	0.194 [0.194–0.194]
TD3_Phys	36	0.000 [0.000–0.000]	0.000 [0.000–0.000]	0.194 [0.194–0.194]

than other methods though it takes more drastic maneuvers, throwing it slightly more off course. It takes slightly more time for it to fully recover. This however, counterbalances the time saved from no longer needing to accelerate back to the same speed. We approximate as below assuming a constant acceleration a typical of the vehicle we chose.

$$T_{recovery} = T_{slip} + \frac{V - v_{exit}}{a} \tag{14}$$

We use $V = 100$, though the relative difference between models remains for any V. We find that using an extrapolated $a = \frac{100\frac{km}{h}}{3.6s}$, the scores obtained can be found in Table II. We see that while the $TD3_{Phys}$ slightly lags behind the SAC benchmark, it outperforms the base $TD3_{60k}$ significantly with overall recovery time being around 20.66% faster.

D. Out of Distribution Failure

For failure analysis, we report three component severities and an aggregate crash severity. Crash severity metrics are computed exactly as defined in the terminal reward (normalized/clipped, squared components with a weighted sum).

1) Aggregate Out of Distribution Analysis: As shown in Table III, SAC and TD3_60k exhibit larger typical slip-related contributions than TD3_Phys on OOD_COMBINED_HARD. However, this difference should be interpreted primarily as an outcome of robustness. TD3_Phys succeeds on many episodes where the other agents fail, resulting in a high fraction of near-zero slip and gap contributions in the aggregate statistics. In other words, the table indicates that the OOD perturbation induces failures for some methods more frequently, rather than implying that one method necessarily "fails more gently" than another.

As a result, cross-agent differences in the component summaries under the per-agent failures-only view combine two effects: (i) differences in *failure mode* (how severely an agent fails) and (ii) differences in *episode selection* (which scenarios

TABLE IV
WEIGHTED COMPONENT SEVERITY EXACTLY AS IN THE TERMINAL REWARD ON THE MATCHED HARD-SET $\mathcal{H}$ FOR OOD_COMBINED_HARD (EPISODES WHERE TD3_60K FAILS). EACH ENTRY REPORTS MEDIAN AND INTERQUARTILE RANGE (Q1–Q3).

Agent	n	$1.2\cdot$ slip	$0.5\cdot$ gap	$0.25\cdot$ speed
SAC	17	0.051 [0.003–0.279]	0.087 [0.057–0.114]	0.194 [0.194–0.194]
TD3_60k	17	1.036 [0.360–1.125]	0.055 [0.050–0.176]	0.194 [0.194–0.194]
TD3_Phys	17	0.000 [0.000–0.030]	0.000 [0.000–0.123]	0.194 [0.194–0.194]

the agent fails on). To disentangle these, we additionally report a matched hard-set analysis, where we fix an episode subset $\mathcal{H}$ defined by the failures of a chosen reference agent and evaluate *all* agents on the same episode IDs. This controls for variation in the episode mixture and enables a fairer cross-agent comparison of severity and component contributions under a shared set of reference-hard scenarios.

2) Matched Hard Set Analysis: In addition to standard *failures-only* summaries computed separately for each agent, we also report results on a *matched hard-set* defined by a reference baseline. Conditioning on each agent's own failures can yield comparisons over different sets of episodes: one agent may fail primarily on extremely difficult scenarios, while another may fail on a different mixture of cases. As a result, differences in failure severity may be confounded by differences in the underlying distribution of episode difficulty rather than differences in recovery behavior.

To control for this effect, we define a hard-set as the subset of evaluation episodes for which a chosen reference agent fails. In our experiments, we use TD3_60k as the reference baseline and define

$$\mathcal{H} = \{e : \text{TD3\_60k fails on episode } e\}. \tag{15}$$

Evaluated on this same subset, TD3_Phys successfully recovers $10/17$ episodes (58.8%), whereas TD3_60k succeeds on $0/17$ by construction. We then analyze terminal crash severity and its weighted component contributions on $\mathcal{H}$. By holding the episode subset fixed, this analysis ensures that all agents are evaluated on scenarios of the *same underlying difficulty* (as defined by the reference agent's failures), rather than on different mixtures of easier and harder OOD episodes. Importantly, while the scenarios are matched, the *agents' outcomes need not be*: an episode where TD3_60k fails may still be recovered successfully by another agent (or vice versa). This allows us to attribute differences in performance more to cross-agent differences in recovery behavior, and not to differences in the distribution of relative scenario difficulty.

Table IV summarizes the weighted terminal-reward components on the matched hard-set $\mathcal{H}$ (episodes where TD3_60k fails). Two observations stand out. First, the speed component is essentially constant across agents, indicating that cross-agent differences on $\mathcal{H}$ are not driven by speed variation. Second, the slip component shows the strongest separation: TD3_60k's failures on $\mathcal{H}$ are slip-dominated (median 1.036), whereas TD3_Phys exhibits near-zero median slip (and similarly low gap), consistent with recovering many of the same episodes rather than accumulating large terminal costs.

This decomposition suggests that TD3_Phys's gains are driven by improved slip stabilization rather than by changes in speed, and that residual failure cost, when they occur tend to shift away from slip-dominated outcomes.

Overall, these results indicate that physics-informed TD3 improves recovery on challenging OOD scenarios by better controlling slip dynamics, translating into fewer failures on a fixed set of reference-hard episodes and a qualitatively different distribution of terminal cost contributions.

VII. CONCLUSION AND FUTURE WORK

This work presented *BeyondGrip*, a physics-informed reinforcement learning framework designed to recover racecars from high-slip instability. By integrating vehicle dynamics—specifically tire slip limits and load transfer—into the observation space and reward structure, we demonstrated that RL agents can learn robust recovery policies that generalize beyond their training conditions.

Our experiments reveal a critical trade-off: while standard algorithms like SAC achieve high performance in nominal conditions, they often fail brittlely under stress. In contrast, our physics-informed TD3 agent demonstrates superior generalization to out-of-distribution velocities and friction coefficients. It significantly reduces crash severity and maintains control in scenarios where baseline methods experience catastrophic failure. These results suggest that embedding physical domain knowledge is essential for developing autonomous racing agents that are not only fast but safe under unpredictable conditions.

Future work will focus on integrating these recovery behaviors into a hierarchical control architecture. Specifically, we aim to develop a supervisor that dynamically switches between a relative-action controller for precision racing and our absolute-action recovery policy when stability limits are exceeded, combining the optimality of standard RL with the safety guarantees of physics-informed control

REFERENCES

[1] Chayan Banerjee et al. "A survey on physics informed reinforcement learning: Review and open problems". In: *Expert Systems with Applications* 287 (Aug. 2025), p. 128166. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2025.128166. URL: http://dx.doi.org/10.1016/j.eswa.2025.128166.

[2] Axel Brunnbauer. "Model-based deep Reinforcement learning for autonomous racing". Diploma Thesis. Technische Universität Wien, 2021. DOI: 10.34726/hss.2021.86588. URL: https://doi.org/10.34726/hss.2021.86588.

[3] Hikaru Hoshino et al. *Autonomous Drifting Based on Maximal Safety Probability Learning*. 2024. arXiv: 2409.03160 [cs.RO]. URL: https://arxiv.org/abs/2409.03160.

[4] Timothy P. Lillicrap et al. *Continuous control with deep reinforcement learning*. 2019. arXiv: 1509.02971 [cs.LG]. URL: https://arxiv.org/abs/1509.02971.

[5] Adrian Remonda et al. *A Simulation Benchmark for Autonomous Racing with Large-Scale Human Data*. 2024.

[6] G. E. Uhlenbeck and L. S. Ornstein. "On the Theory of the Brownian Motion". In: *Phys. Rev.* 36 (5 Sept. 1930), pp. 823–841. DOI: 10.1103/PhysRev.36.823. URL: https://link.aps.org/doi/10.1103/PhysRev.36.823.

[7] Yuanda Wang, Xin Yuan, and Changyin Sun. "Learning autonomous race driving with action mapping reinforcement learning". In: *ISA Transactions* (May 2024). ISSN: 0019-0578. DOI: 10.1016/j.isatra.2024.05.010. URL: http://dx.doi.org/10.1016/j.isatra.2024.05.010.

VIII. Appendix

A. Definitions

$$r_{base} = 30$$
$$r_{time}(T) = 10(1 - \tfrac{T}{350})$$
$$r_{speed}(v) = 5\tanh(\tfrac{v}{30})$$
$$r_{gap}(d) = 15e^{-\frac{\mathrm{d}}{1.5}}$$

$$a = 1.9 + b(s, d),$$
$$b(s,d) = \begin{cases} 3, & s < 1.5 \text{ and } d < 2, \\ 1.5, & s < 1.5 \text{ and } 2 \le d < 4, \\ 0, & \text{otherwise.} \end{cases}$$

$$i = 6 * \text{clip}(2\Delta d, -1, 1)$$
$$r_{min} = -8$$
$$r_{max} = 2$$

Physics Rewards:

$$S_{os} = \frac{|\alpha_r|}{|\alpha_f| + |\alpha_r| + \epsilon}$$

$$r_{load} = S_{os} \cdot W_{load} \cdot \left(\frac{F_{z,r} - F_{z,f}}{F_{z,r} + F_{z,f}}\right)$$

$$r_{slipratio} = S_{os} \cdot W_{slip} \cdot \exp\left(-500 \cdot (\bar{\kappa}_r - 0.015)^2\right)$$

$$\delta_{target} = \text{clip}(\alpha_r, -\delta_{max}, \delta_{max})$$
$$r_{align} = S_{os} \cdot W_{steer} \cdot \exp\left(-100 \cdot (\delta - \delta_{target})^2\right)$$

$$r_{compliance} = \begin{cases} W_{steer} \cdot \tanh\left(\frac{P_{steer}}{\tau}\right) & \text{if } P_{steer} > 0 \\ -W_{steer} \cdot \tanh\left(\frac{|P_{steer}|}{\tau}\right) & \text{if } |M_z| > M_{th} \\ 0 & \text{otherwise} \end{cases}$$

Penalties

$$p_{base} = 20$$
$$p_{severity}(s, d, v) = (1.2s^2 + 0.5d^2 + 0.25v^2)$$
$$p_s = \min(0.8s^2, 20) * \text{clip}(\frac{\overbrace{450 - T}^{\text{time decay}}}{450 - 100}, 0, 1)$$
$$p_g = \min(0.1d^2, 15) * \text{clip}(\tfrac{450-T}{450-100}, 0, 1)$$

Other Parameters

$$\alpha = \tfrac{1}{150}$$
$$\beta = 0.2$$

B. Soft Actor Critic

The first algorithm we used is Soft Actor Critic (SAC). As opposed to standard Policy Gradient methods, it uses the measure of entropy to heuristically measure simplicity, we expect simpler models to be more correct as they likely, under assumptions similar to those of Occam's Razor, better expand to universal laws. As a side-effect, this also encourages exploration.

$$\mathcal{H}(\pi(\cdot|s_t)) = \mathbb{E}_{a_t \sim \pi(\cdot|s_t)}\left[-\log \pi(a_t|s_t)\right]$$

Our new model is now the sum of the reward and the entropy weighted with hyperparameter α

$$\pi^* \in \arg\max_{\pi} \mathbb{E}_{\tau \sim P_\theta(\cdot)}\left[\sum_{t=0}^{\infty} \gamma^t (\mathcal{R}(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t)))\right]$$

The SAC algorithm also is an actor-critic method. First, instead of treating $\mathcal{R}(s, a)$ as a black box, or future rewards over multiple time steps like it is indivinable, unknowable, it estimates the future return using learning. These methods approximate future returns using some combination of

$$V_\phi(s), Q_\phi(s, a), A_\phi(s, a)$$

Since this change into a white box, there is actually now an additional derivative path of θ through this function. We recall that a is a sample from π_θ, through the reparameterization trick, we can express a as some function of externally created noise ϵ:

$$a = f_\theta(s, \epsilon)$$

We differentiate with respect to θ through this $A_\phi(s, f_\theta(s, \epsilon))$ term.

Other small changes in the SAC method include using two Q_{ϕ_1}, Q_{ϕ_2} functions, with the minimum of the two used to avoid overfitting. Additionally, replay buffers are used for the Q functions so that they do not forget values of off-policy/rare states. These are learned nearly identically to the learning steps in the following algorithms.

The final algorithm combines these ideas and a number of mathematical intermediate steps like simplifying the derivative over the new objective that are derived in more detail in the original work.

C. Twin Delayed DDPG

The above approach suffers in that it often allows for the overestimation of Q values leading to training instability. Instead, the authors of this work (TD3) offer that a few minor changes to the existing work should be made.

One, clipped added noise should be used to smooth the distribution of the policy; this regularization technique forces the model to treat similar actions more uniformly, so no large spikes which was a common issue of the DDPG algorithm.

Two, delaying policy updates by only updating it once for every two Q function updates.

Three, learning two Q functions instead of one exactly in the same way that SAC does it.

D. State Space

The observation space includes vehicle-level signals such as longitudinal speed, lateral deviation from the reference trajectory, engine RPM, acceleration, selected components of local and angular velocity, and a down sampled look ahead of track curvature. These variables provide global context regarding motion and trajectory tracking while remaining compatible with standard racing telemetry.

To explicitly expose slip-related dynamics, the state incorporate per-wheel slip angles for all four tires, which serve as the primary indicators of oversteering and understeering behavior. In addition, we include physics-informed signals such as lateral tire grip coefficients and per-wheel vertical load, which reflect changes in tire-road interaction during aggressive maneuvers. Vehicle orientation (yaw, pitch, and roll) is also provided to enable the agent to infer heading misalignment and transient instability during recovery.